Research Problem: Sparse Access, Large State

- MoE 每次只激活少数专家，但全部专家都要随时可访问。
  专家权重可能放在 GPU 显存、CPU 内存或 SSD。
  选中 SSD 上的专家会产生读取等待；MECC 要减少这种等待。



关键代价：被路由选中的冷专家必须从 SSD 读取，生成因此等待。

Related Work: Deployment Gaps

- 现有系统已能跨显存、内存和 SSD 按需执行专家，并在推理时保留常用专家。
  共享服务器还需要回答：给定单任务预算如何安排驻留、如何减少初始冷读、
  以及部署前如何估计 SSD 压力。MECC 将这三项能力纳入同一方案。

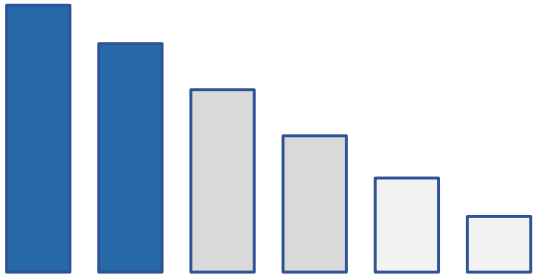
方法	三级存储支撑	完整专家计算	按预算生成驻留方案	历史热点预热	冷读压力预估	在线适应热点变化
SwapMoE[1]	✗	✗	✗	✓	✗	✓
Fiddler[2]	✗	✓	✗	✗	✗	✗
MoE-Infinity[3]	✗	✓	✗	✗	✗	✓
MoE-Lightning[4]	✗	✓	✗	✗	✗	✗
SSD-LLaMA[5]	✓	✓	✗	✗	✗	✓
MECC	✓	✓	✓	✓	✓	✓

MECC: Compile Once, Adapt Online

- MECC 用两种时间尺度解决三级专家驻留问题：部署前按访问画像与预算确定初始位置，运行期随真实路由更新显存和 CPU 缓存。这种协同在 A100 Mixtral 的完整推理中得到验证。

编译期：预算放置

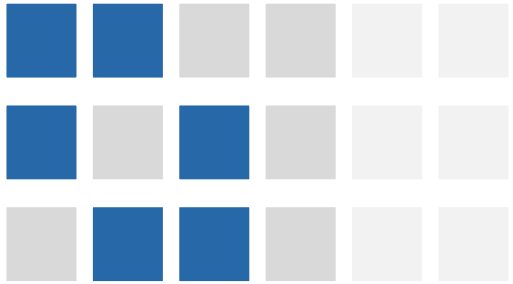
路由热度（示意）



高频 → 快层；低频 → SSD

运行期：联合保留

驻留轨迹（示意）



显存 / CPU 随访问联合更新

真机：端到端验证

相对综合性能



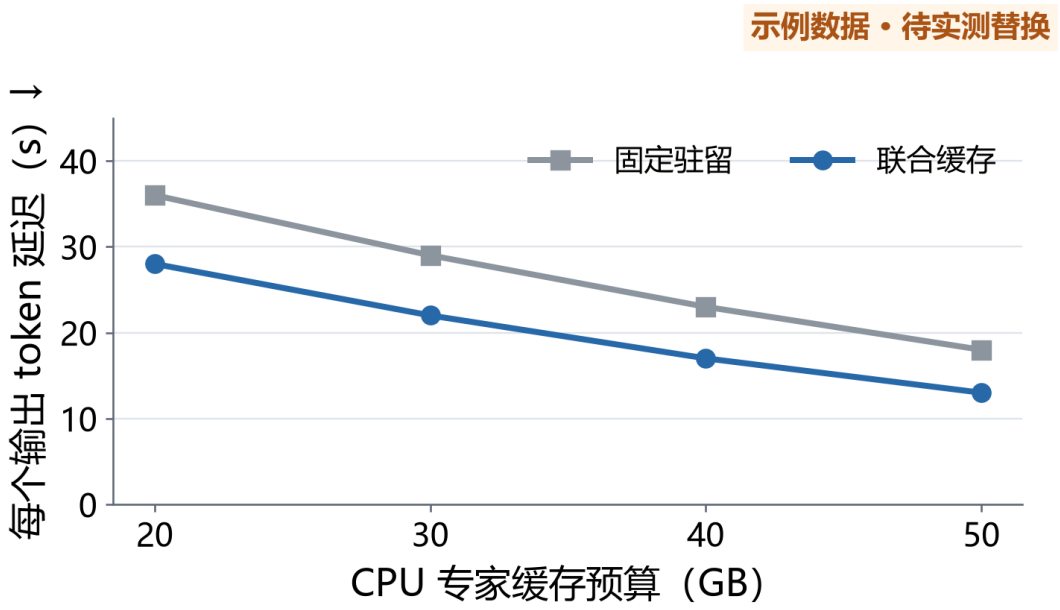
A100 • Mixtral-8x7B

核心贡献：用静态先验减少初始冷读，用在线反馈保留真实热点。

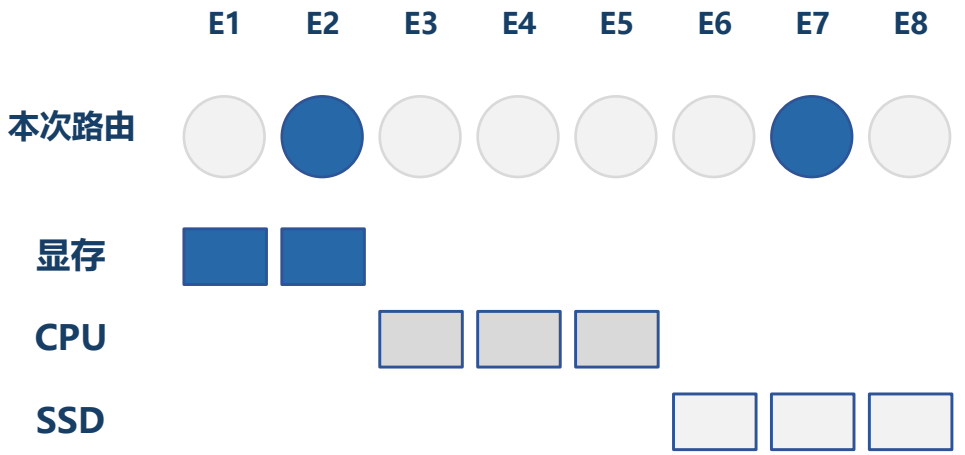
Motivation I: Cold Reads Stall Decoding

- 单任务缓存预算决定有多少专家访问需要进入 SSD；冷读越频繁，生成就越容易等待权重。
  预算扫描用于观察端到端延迟随 CPU 容量的变化，并检验联合缓存能否在同预算下减少等待。

(a) 缓存预算与解码延迟



(b) 稀疏路由与专家驻留位置



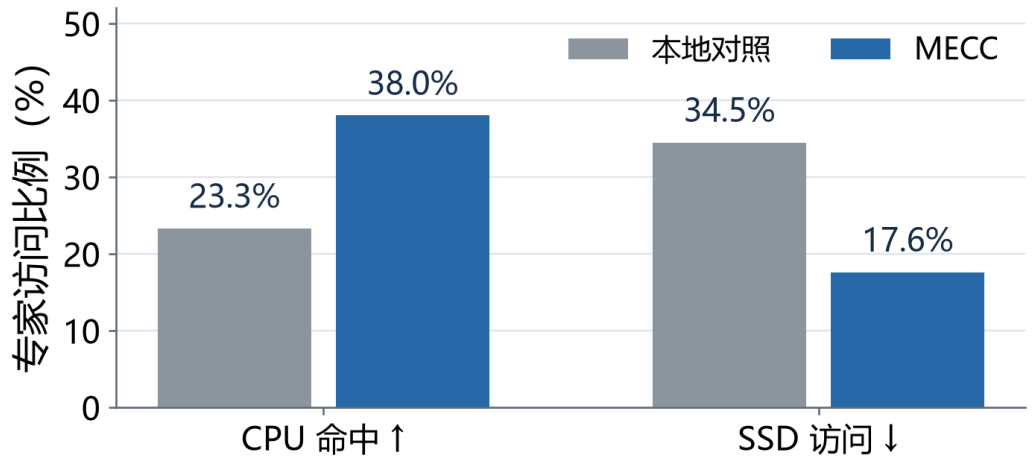
E7 被选中：必须读取 SSD

动机 ①：快层空间有限，需要把容量用在更可能被访问的专家上。

Motivation II: Eviction Breaks Reuse

- 历史 Mixtral A100 实测中，本地对照的 SSD 访问为 34.5%，MECC 为 17.6%。
  CPU 命中同时由 23.3% 增至 38.0%；这一结果提示，显存淘汰后的专家仍值得在 CPU 层保留。

(a) 相同预算下的缓存访问分布



(b) 专家淘汰后的跨层复用



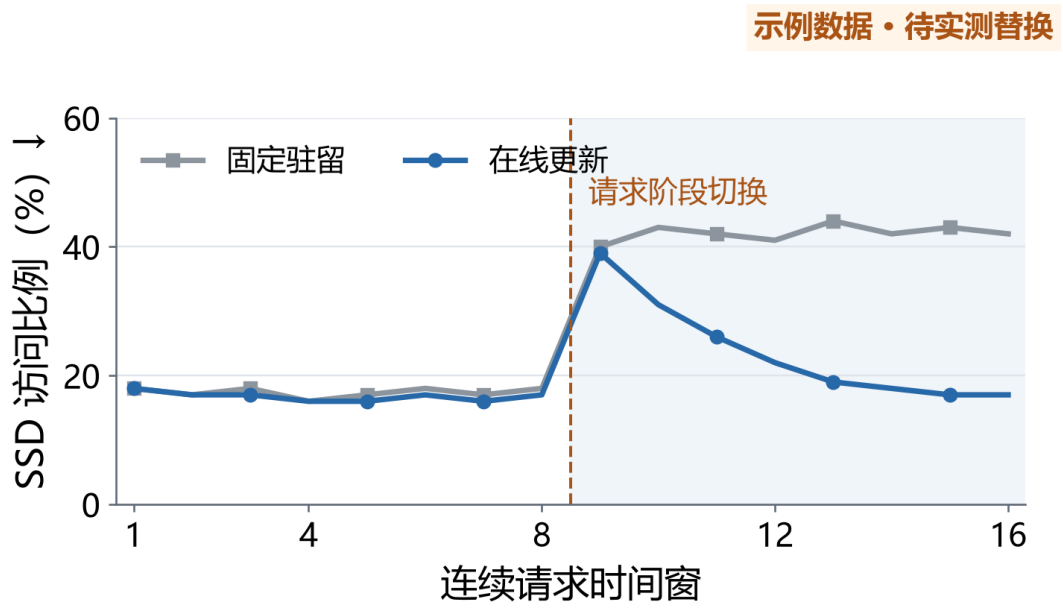
机制示意：再访时由 CPU 承接

动机 ②：显存与 CPU 应共同保留工作集，让复用跨越显存淘汰事件。

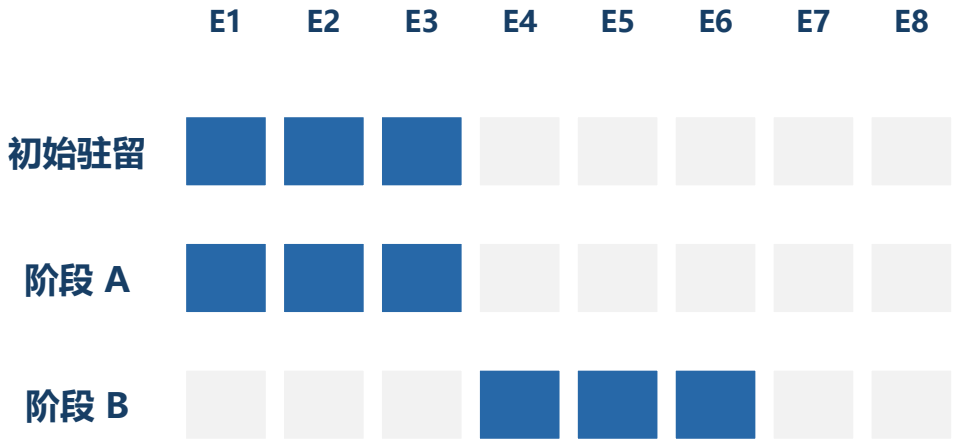
Motivation III: Profiles Become Stale

- 请求阶段发生变化时，历史画像中的高频专家可能不再覆盖当前热点。
  按时间窗观察 SSD 访问，能够区分“固定驻留持续失配”与“在线缓存逐步适应”两种行为。

(a) 请求阶段切换后的冷读压力



(b) 新热点与初始驻留失配

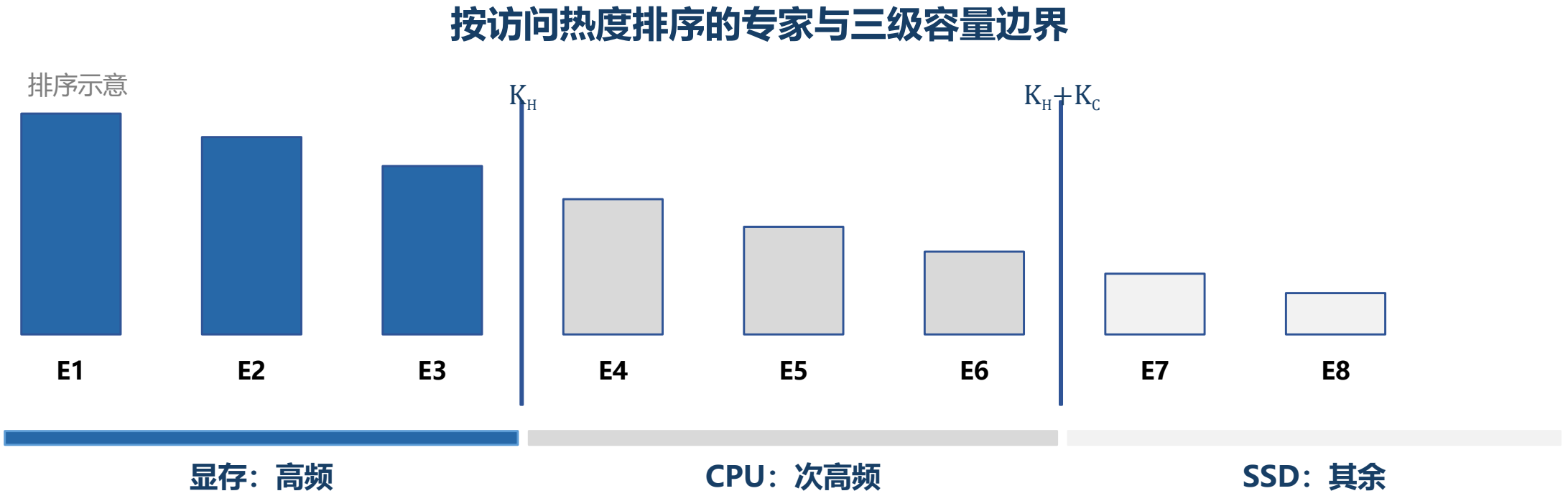


机制示意：新热点仍在原快层之外

动机 ③：部署前画像给出起点，运行时反馈负责持续纠偏。

Insight I: Place by Expected Access Cost

- 洞察 ①：不同层级的取数代价高度不对称，应按访问概率与单任务预算共同决定初始驻留。把高频专家放入显存、次高频专家放入 CPU，剩余留在 SSD；目标是降低预期取数代价，而不是平均分配空间。

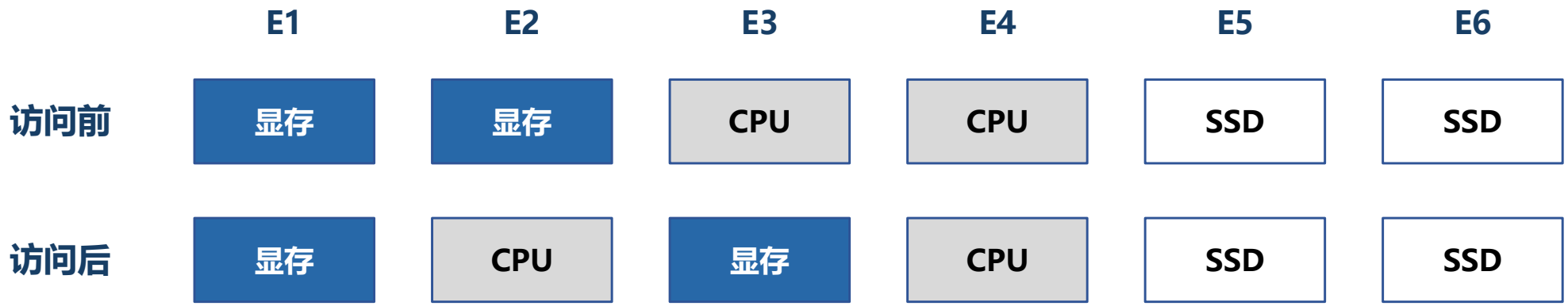


对应动机 ①：优先避免高概率请求承担昂贵的 SSD 冷读。

Insight II: Retain Experts Across Evictions

- 显存淘汰的专家可在 CPU 层继续保留，CPU 命中后再提升到显存。
  这样，专家复用能够跨越显存淘汰；两层共同保存近期工作集，减少再次读取 SSD。

一次 E3 访问前后：各专家所在的最快层级



E3 从 CPU 提升至显存；被替换的 E2 留在 CPU

对应动机 ②：让专家复用跨越显存淘汰事件。

Insight III: Combine Prior and Feedback

- **历史画像决定“从哪里开始”，真实路由决定“接下来保留谁”。**
  编译期给出初始驻留；当热点改变，运行期用最近访问修正快层内容，使位置重新覆盖当前热点。



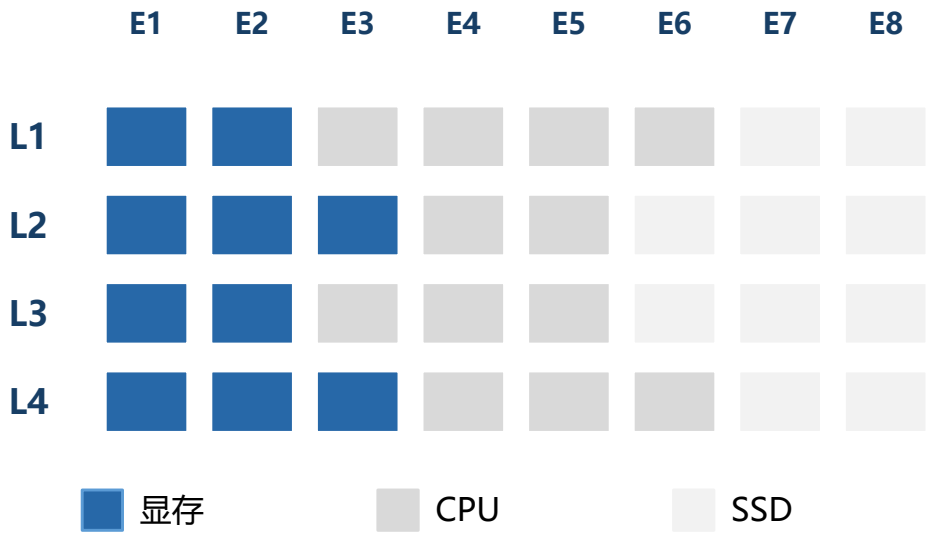
深色单元格表示当前热点或快层保留位置；专家身份与阶段均为机制示意。

洞察 ③：把离线先验和在线反馈放在两个时间尺度上协同使用。

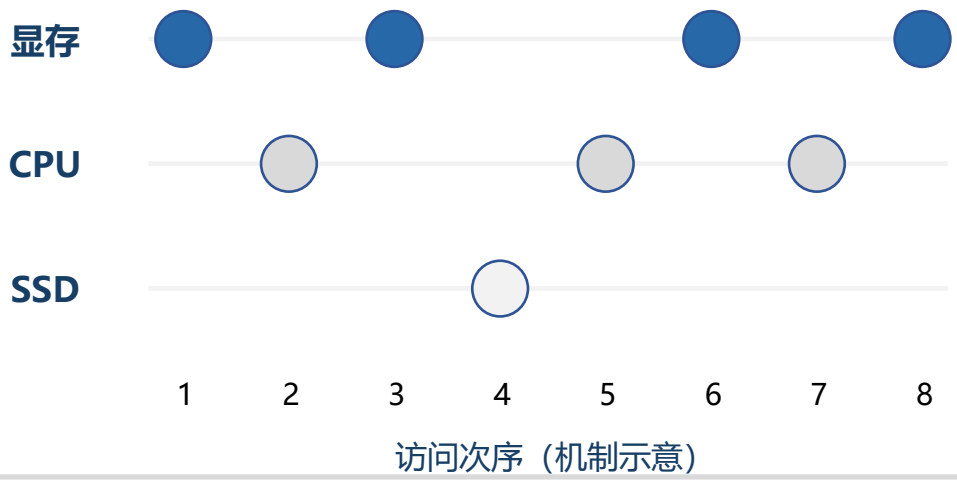
System Overview: From Prior to Feedback

- 编译期把访问画像与单任务预算转成逐层驻留方案；运行期根据真实路由更新显存与 CPU 缓存。两部分共享同一个三级存储视图：SSD 保存完整权重，快层位置随专家访问改变。

编译期：专家 × 层级的初始驻留



运行期：每次访问实际命中哪一层

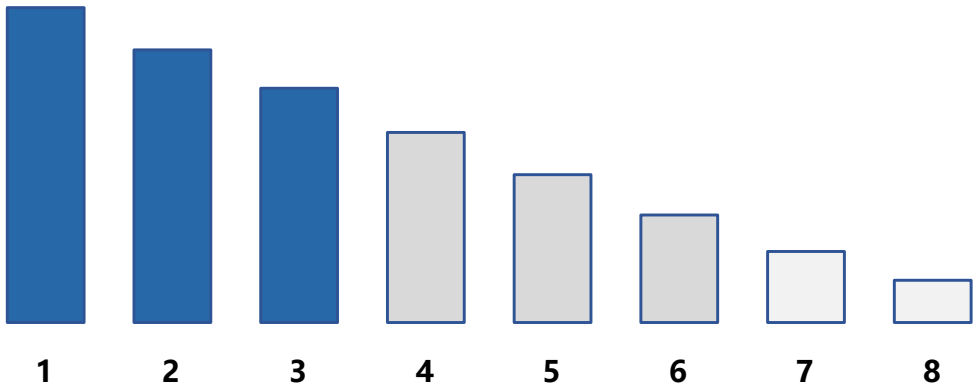


编译期确定初始位置；运行期通过显存与 CPU 的联合保留持续纠偏。

Method I: Budget-Constrained Placement

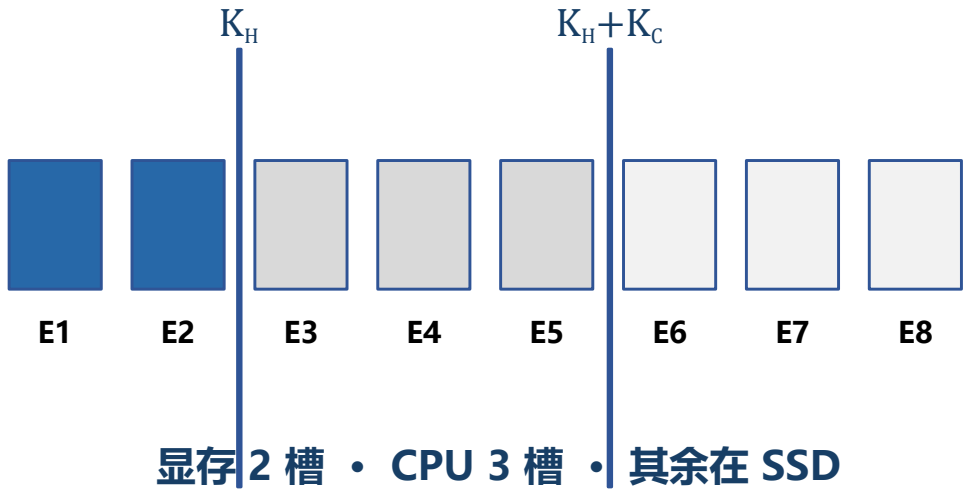
- 编译器读取各层专家的访问画像、权重大小与单任务显存/CPU 预算。在给定独立容量上限下，按访问概率排序并确定两道容量边界，输出每层专家的初始三级驻留位置与预期冷读比例。

访问画像：专家访问频率（示意）



专家按访问概率降序排列

编译产物：预算约束下的三级分配

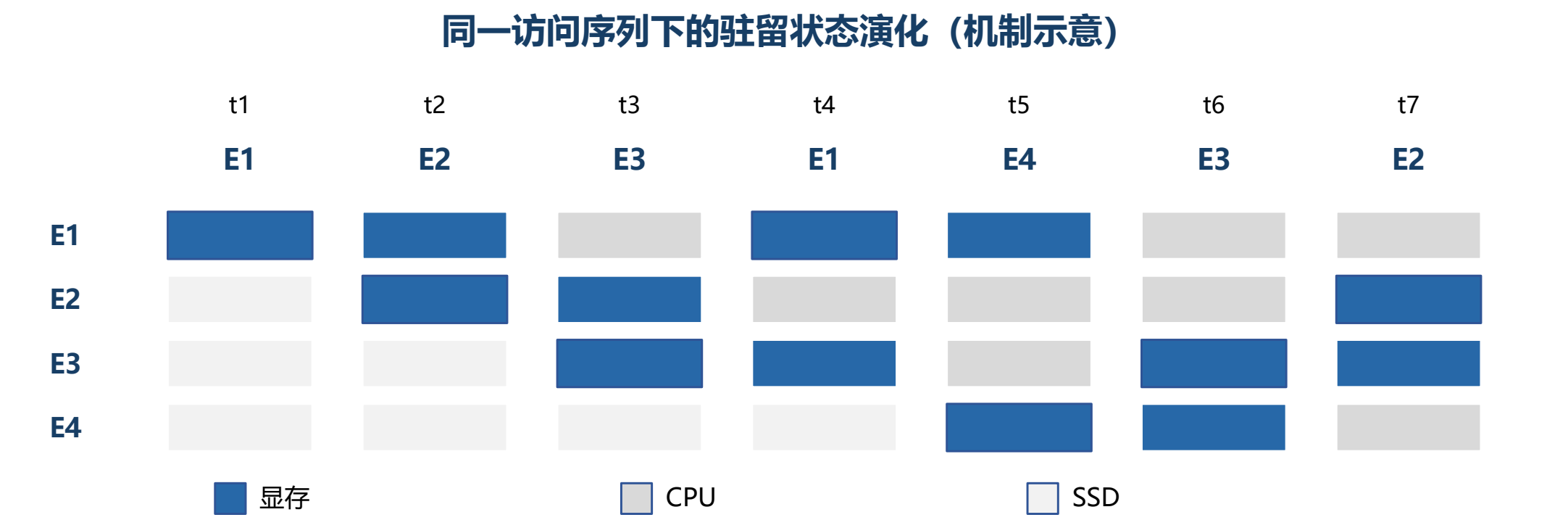


优化目标：最小化预期取数代价

$$\mathbb{E}[T] = p_H T_H + p_C T_C + p_S T_S$$

Method II: Joint HBM–CPU Retention

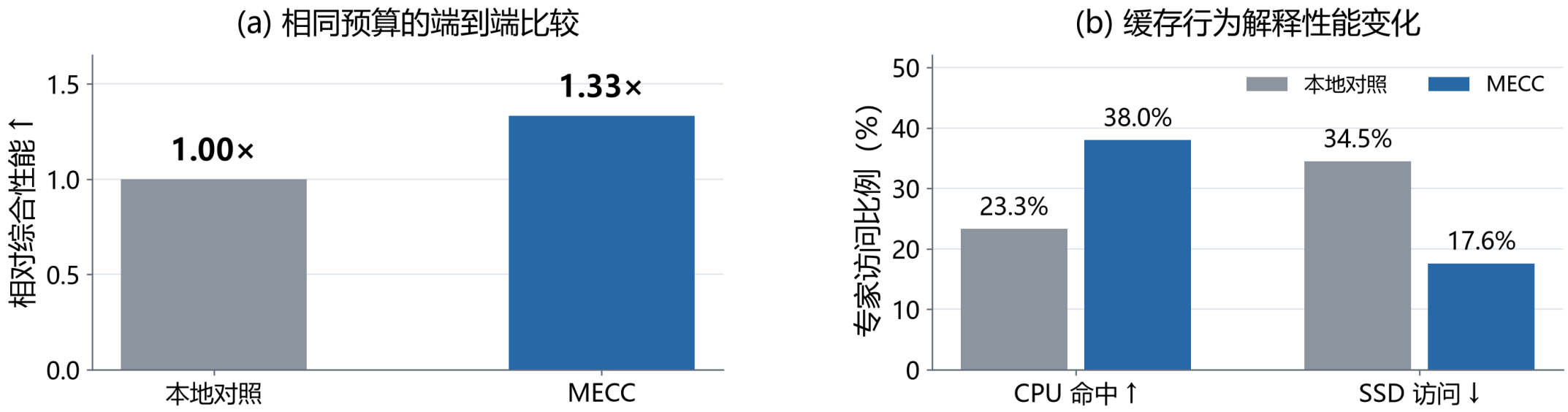
- 真实路由驱动专家访问：显存命中直接复用，CPU 命中提升到显存，两层未命中才读取 SSD。下图以显存 2 槽、CPU 2 槽演示联合保留；每列表示一次访问完成后各专家的最快驻留层级。



E1 与 E2 再访时由 CPU 承接；每次迁移均保持原专家权重与路由选择。

Evaluation: Faster with Fewer Cold Reads

- **历史 A100 Mixtral 主实验使用相同的 50 GB 显存与 40 GB CPU 预算。**
  MECC 的综合性能达到本地 SwapMoE-style 对照的 1.33×，并将更多访问转移到 CPU 缓存。



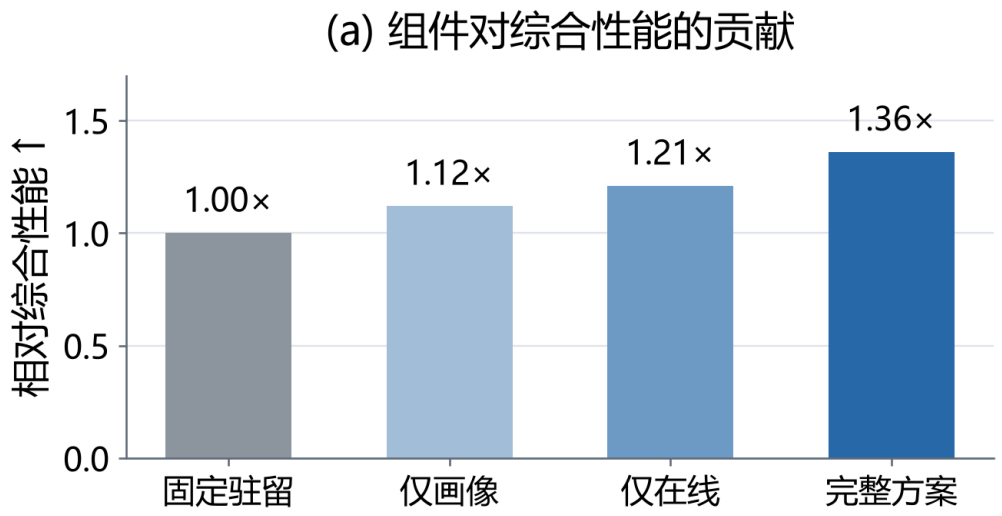
D2 真机 · Mixtral-8x7B · 来源：mixtral_real.json（M1 / M3 / M4）

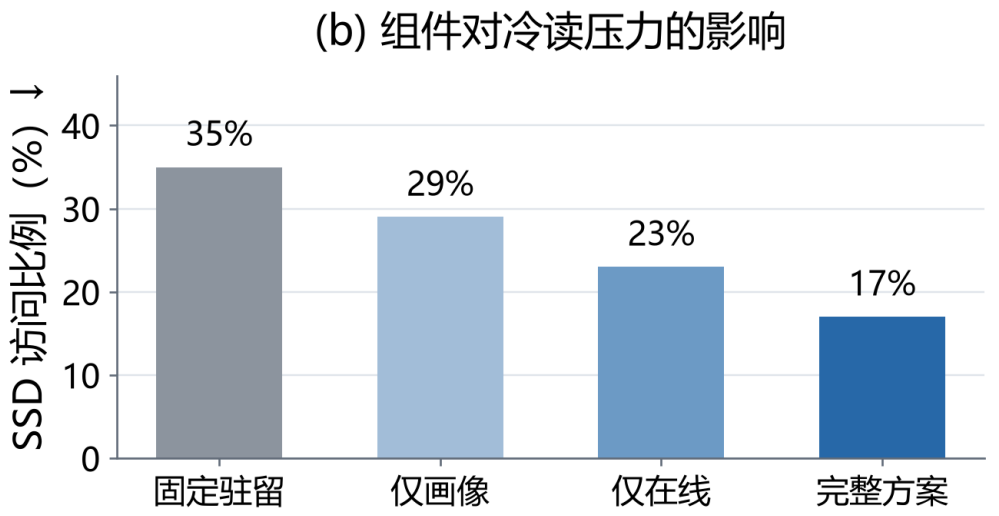
主结果：同预算下提高综合性能，同时减少进入 SSD 的专家访问。

Ablation: Placement Meets Online Retention

- 组件消融分别保留画像引导、在线更新及两者组合，使用相同的模型、输入与缓存预算。
  同时比较综合性能与 SSD 访问，检验性能收益能否由缓存行为的变化解释。

示例数据 • 待实测替换





MECC：用编译期先验安排起点，用运行期反馈保留真实工作集。